Pre-requisites

EduGenie - AI Learning Assistant

1. Purpose of this Document

EduGenie is a multi-model AI learning portal. It combines a FastAPI web server, a small HTML/CSS front end, a local Hugging Face model and the Gemini cloud API. Because several technologies work together, a developer needs the right knowledge, software, accounts and hardware before starting. This document lists each requirement, explains *why* it is needed in this project, and shows how to prepare the environment step by step.

2. Knowledge Pre-requisites

The table links each skill to the exact place where it is used in the EduGenie code, so a new team member knows what to study first.

Skill	Where it is used in EduGenie	Why it matters
Python 3	All modules (Main.py, qna.py, quiz_module.py ...)	Functions, type hints, Enum, exceptions and environment variables are used throughout.
FastAPI	Main.py: app, @app.get, @app.post, Form(...)	Defines the two routes and receives the form data sent by the browser.
Jinja2 templating	templates/index.html: {{ response }}, {% if response %}	Lets the server insert the answer, the prompt and the model name into the page.
HTML and CSS	index.html and static/style.css	Builds the form, the answer box and the visual styling.
Prompt writing	Every module builds a formatted_prompt	The quality and layout of each answer depend on clear instructions to the model.
Hugging Face Transformers	explanation_module.py	Loads a tokenizer and a T5 model and generates text on the CPU.
REST/API basics	google-genai client in Main.py	The Gemini features call a cloud API with a secret key.
Command line and Git	Running uvicorn, pip, version control	Needed to install, run and share the project.

3. Software Requirements

All Python packages are listed in requirements.txt and are installed with one command. The 'Role in EduGenie' column explains what each package actually does in the application.

Package	Version	Role in EduGenie
Python	3.9 or higher	Runs every module of the application.
fastapi	>= 0.110.0	Web framework: routing, form handling, template responses.
uvicorn	>= 0.28.0	ASGI server that listens for browser requests and runs the app.
jinja2	>= 3.1.3	Renders index.html with the values passed from Main.py.

Package	Version	Role in EduGenie
python-multipart	>= 0.0.9	Required by FastAPI to read HTML form fields (prompt, task).
python-dotenv	>= 1.0.1	Reads GEMINI_API_KEY from the .env file so the key is not hard-coded.
google-genai	>= 0.1.1	Official client used to call the Gemini model.
transformers	>= 4.38.0	Downloads and runs the LaMini-Flan-T5-783M model.
torch	>= 2.2.0	Deep-learning engine that executes the local model.
sentencepiece	>= 0.2.0	Tokenizer library needed by T5-family models.
protobuf	>= 4.25.0	Supporting library required by the tokenizer and model loading.

4. Accounts, Keys and Network Access

- **Gemini API key (Google AI Studio):** Main.py reads GEMINI_API_KEY (or GOOGLE_API_KEY). If neither exists the program raises a RuntimeError and refuses to start, so the problem is found immediately rather than when a student submits a question.
- **Key type check:** a value beginning with "ya29." is an OAuth access token, which the Gemini client does not accept. The code detects this case and explains that an AI Studio API key is needed.
- **Internet for the first run:** the local model (about 783 million parameters) is downloaded from Hugging Face the first time the Concept task is used, and then reused from the local cache.
- **Internet during use:** the four Gemini tasks (Q&A, summary, quiz, learning path) need a connection on every request.

5. Hardware Requirements

Component	Minimum	Recommended	Reason
CPU	Dual-core	Quad-core or better	The local model runs on CPU (no GPU code in the project).
RAM	4 GB	8 GB or more	The model and PyTorch are held in memory after the first concept request.
Disk	4 GB free	6 GB or more	PyTorch, Transformers and the downloaded model files.
GPU	Not required	Optional	The code does not use one, so it is a possible future speed-up.

These hardware figures are planning estimates, not measurements. The real download and memory size depend on the PyTorch build installed on your machine.

6. Environment Preparation (Step by Step)

```
1. Check Python:              python --version
2. Create virtual environment: python -m venv venv
3. Activate it:               venv\Scripts\activate     (Windows)
                              source venv/bin/activate    (Linux / macOS)
```

```
4. Install packages:          pip install -r requirements.txt
5. Create the .env file:      GEMINI_API_KEY=your_api_key_here
6. Check folder layout:       Main.py, modules, templates/index.html, static/style.css
7. Start the server:          uvicorn Main:app --reload
8. Open the application:      http://127.0.0.1:8000
```

Why a virtual environment? PyTorch and Transformers are large and version-sensitive. A virtual environment keeps them separate from other projects and makes the setup repeatable on another computer.

Why a .env file? It keeps the secret API key outside the source code. Main.py loads it from the same folder as the script using Path(__file__).resolve().parent, so it is found no matter where the command is started. The file must never be uploaded to a public repository.